

Variable Number of Arguments in C++

In C++, functions can be designed to accept a variable number of arguments of any type or count.

This is achieved through **variadic templates** (introduced in C++11) and **fold expressions** (introduced in C++17).

These features are modern, type-safe replacements for old-style variable argument methods such as `printf()` and `varargs`.

1) The `sizeof...` Operator

Syntax

```
sizeof...(args)
```

Meaning

- `sizeof...` gives the number of elements in a parameter pack.
- It works only with:
 - Template parameter packs, for example: `template<typename... T>`
 - Function parameter packs, for example: `void fun(T... args)`

Invalid Usage

`sizeof...` cannot be applied to a normal variable such as an array.

Example:

```
int arr[] = { 10, 20, 30, 40 };  
cout << sizeof...(arr); // Error: arr is not a parameter pack
```

2) Variadic Templates (C++11)

A **variadic template** allows a function or class to accept a variable number of arguments of different types.

General form:

```
template <typename T, typename... Args>  
void function(T first, Args... args);
```

Explanation:

- `T first` represents the first argument.
 - `Args... args` represents the remaining arguments, known as a parameter pack.
-

3) Example: Recursive Variadic Function

```
#include <iostream>  
using namespace std;  
  
// Base case: no arguments remaining  
void disp() {  
    cout << "terminating now" << endl;  
}  
  
// Recursive case: process first argument, then recurse  
template <typename T, typename... Args>  
void disp(T first, Args... args) {  
    cout << first << endl;  
    disp(args...);  
}
```

```

    disp(args...);
}

int main() {
    int i = 1;
    double d = 3.4;
    char str[] = "hello";
    char c = 'A';
    disp(i, d, str, c);
    return 0;
}

```

Output

```

1
3.4
hello
A
terminating now

```

4) Step-by-Step Explanation of the Process

1. First Call:

```

disp(i, d, str, c);
T = int
Args... = double, const char[6], char

```

2. Second Call:

```

disp(d, str, c);
T = double
Args... = const char[6], char

```

3. Third Call:

```

disp(str, c);
T = const char[6]
Args... = char

```

4. Fourth Call:

```

disp(c);
T = char
Args... = (empty)

```

5. Base Case Call:

```
disp();
```

- No arguments remain — recursion terminates.

5) Base Case Requirements

The base case function must be **parameterless**, for example:

```
void disp() { ... }
```

Reason:

- The recursive calls eventually reach a point where there are no arguments left.
 - If the base case had parameters (e.g. `void disp(int n)`), the compiler would not find a match when the argument pack becomes empty.
-

6) Return Type of the Base Function

- Technically, the base function can return any type.
 - However, it is **recommended to use void**, as returning a value from the base case serves no practical purpose.
-

7) Comparison with Java's Varargs

Java Example

```
void disp(Object... args) {  
    for (Object k : args)  
        System.out.println(k);  
}  
  
disp(1, 3.4, "hello", 'A', new Thread());
```

Equivalent Concept in C++

C++ implements this behavior using **variadic templates**:

```
template <typename... Args>  
void disp(Args... args);
```

Each argument may have a different type, which the compiler deduces automatically.

8) Parameter Packs Explained

A **parameter pack** is a placeholder for zero or more template arguments.

- **Template parameter pack:**
`template<typename... Args>` – represents a list of types.
- **Function parameter pack:**
`void disp(Args... args)` – represents a list of values.

Example:

```
disp(1, 2.5, "hello", 'A');
```

Internally, the compiler expands this to:

```
disp<int, double, const char*, char>(1, 2.5, "hello", 'A');
```

9) Fold Expressions (C++17)

C++17 introduced **fold expressions**, which eliminate the need for recursion in variadic templates.

Example:

```
#include <iostream>
```

```

using namespace std;

template <typename... Rest>
void disp(Rest... rest)
{
    cout << "in disp" << endl;
    ((cout << rest << "\t"), ...); // Fold expression
}

int main() {
    int i = 1;
    double d = 3.4;
    char str[] = "hello";
    char c = 'A';
    disp(i, d, str, c);
}

```

Output

```

in disp
1      3.4    hello    A

```

10) Understanding Fold Expressions

General syntax:

```

(expr op ...)    → right fold
(... op expr)    → left fold

```

Purpose:

- Applies an operator (op) across all elements in a parameter pack.

Example:

```
((cout << rest << "\t"), ...);
```

Expands to:

```

(cout << i << "\t"),
(cout << d << "\t"),
(cout << str << "\t"),
(cout << c << "\t");

```

Execution order:

- Left to right.
 - Prints each argument in sequence.
 - Eliminates the need for recursion.
-

11) Advantages of Fold Expressions

1. No recursive calls required.
2. No base case function needed.
3. Code is shorter and easier to read.
4. Compiler automatically unrolls the parameter pack.
5. Cleaner and safer syntax.

12) Example: Counting, Printing, and Summing Arguments

```
#include <iostream>
using namespace std;

template <typename... Args>
auto dispAndSum(Args... args)
{
    cout << "Count = " << sizeof...(args) << endl;
    cout << "Arguments are: ";
    ((cout << args << "\t"), ...); // Fold expression for printing
    cout << endl;

    auto total = (args + ...);    // Fold expression for summing
    return total;
}

int main()
{
    int a = 10;
    double b = 5.5;
    float c = 4.5;

    auto result = dispAndSum(a, b, c);
    cout << "Sum = " << result << endl;
}
```

Output

```
Count = 3
Arguments are : 10    5.5    4.5
Sum = 20
```

13) Using initializer_list for Homogeneous Arguments

When all arguments are of the same type, initializer_list can be used instead of templates.

Example 1

```
#include <iostream>
using namespace std;

template<class T>
void disp(initializer_list<T> args)
{
    cout << "The data sent is of type: " << typeid(T).name() << endl;
    for (T i : args)
        cout << i << endl;
}

int main()
{
    disp({ 10, 20, 30, 40, 50 });
    disp({ "hello", "welcome", "bye" });
    disp({ 4.5, 9.8, 10.3 });
}
```

Example 2

```
#include <iostream>
using namespace std;

void disp(initializer_list<int> args)
{
    for (int i : args)
        cout << i << endl;
}

int main()
{
    disp({ 10, 20, 30, 40, 50 });
}
```

```
    disp({ 1, 2, 3 });  
}
```

Notes:

- `initializer_list` can be iterated using a range-based for loop.
- It works only when all arguments are of the same type.

14) Common Fold Expression Patterns

Purpose	Fold Expression
Print all arguments	<code>((cout << args << "\t"), ...);</code>
Sum all arguments	<code>(args + ...);</code>
Count arguments	<code>sizeof...(args)</code>

15) Summary Table

Concept	Description
Variadic Template	Template that can take a variable number of parameters
Parameter Pack	Represents zero or more arguments
<code>sizeof...(args)</code>	Returns the number of arguments in the pack
Recursive Expansion	Expands the pack one argument at a time
Base Case	Must be parameterless (e.g., <code>void disp()</code>)
Fold Expression	Simplifies expansion of parameter packs in C++17
<code>initializer_list</code>	Container for same-type arguments
Return Type	Typically void, unless a result is needed
Java Equivalent	<code>void disp(Object... args)</code>

16) Conceptual Summary

A fold expression like:

```
((cout << rest << "\t"), ...);
```

can be conceptually read as:

“For each element in the argument pack, print it followed by a tab.”

This expression unrolls automatically at compile time, eliminating the need for recursive calls or a base case.

17) Key Points

1. Variadic templates allow functions to accept arguments of any type and count.
 2. Fold expressions (C++17) simplify code by removing the need for recursion.
 3. `sizeof...(args)` counts the number of arguments in a parameter pack.
 4. The base case in recursion must be parameterless and preferably return void.
 5. `initializer_list` is used when all arguments are of the same type.
 6. These techniques make C++ functions as flexible as Java's Object... arguments, but with type safety and compile-time checks.
-

